m = 3 algebraic decompositions, Curve25519 miniature

Three-term (m = 3) algebraic decompositions, Curve25519 miniature k = 20

Protocol: PROTOCOL.md (frozen before data). Data: `analysis/f4cost_m3.jsonl` (the 20 cells, `run_cells.sh`), `analysis/coverage.jsonl` (exact coverage, no solving). Instrument: `curve25519_family f4cost ... <m> <mode>`; oracle `pkm-semaev:m=3,mode=chain|full[,guess=G]`. Unit: F_p multiply-adds in F4 elimination (not system construction, not the root-grid read, not the lift: those are native work to be priced under AGENTS.md 5a before any cross-method ranking).

Result in one paragraph

No m = 3 cell produced a relation that F4 decided: **0 hits in every cell where F4 ran**. Coverage grows as predicted (~N^3) but the F4 system at m = 3 grows much faster than the coverage: a 9-column base (isogeny t = 4) already costs 1.7e10 multiply-adds per target (the m = 2 cell with a 25-column base costs 5.6e7), and the cells with useful coverage (t >= 6, 25 columns, coverage 0.02) do not fit in 3.5 GB. The best m = 3 per-relation cost, using the exact coverage as the hit rate, is about 1e12 (bits t = 4, guess 3, chain), against about 4e10 for the best m = 2 recomputed the same way (6e9 with the ladder's number). Classification: **regression** (m = 3 is 30x to 200x worse than m = 2; no improvement).

Tests (task 1)

`cargo test --release --lib pkm_semaev`: 8 passed, 0 failed (463 s), in a build directory used by this worktree alone. New tests: `isogeny_tower_m3_agrees_with_grid` (isogeny t = 3), `bit_interval_m3_agrees_with_grid` (bits, 4-point base, guess 0, 1, 2): in each, chain F4, full F4 and the grid agree on 32 targets with `decomposable_points(.., 3, ..)`, relations sum to the target, and the cross-check reports 0 disagreements. No oracle bug found. One test-side observation: on a tiny Boolean system F4 can decide with 0 elimination multiply-adds (32 calls, 0 ops), so the test requires calls > 0 only.

Table (k = 20, 100 targets requested per cell, seeds 2026111300 + t, cap 600 s)

“Coverage” is the exact fraction of the order-r subgroup that is a sum of 3 base points (`decomposable_points`, filtered to points of order dividing r). Per-relation = multiply-adds per target / coverage, because no cell with F4 work has an observed hit (an upper bound of the hit rate, not a measurement). Values are powers of ten (“1e13.2” = 10^13.2).

form	t	G	mode	columns	multiply-adds / target	solving degree	hits / targets	coverage
isogeny	3	0	chain	1	267153396	6	0/100	0
isogeny	4	0	chain	9	17464730752	6	0/100	0.00095
isogeny	5	0	chain	18	4152524735441	7	0/1 (capped)	0.00729
isogeny	6	0	chain	-	failed (memory 3.5 GB / 900 s)	-	-	-
bits	4	0	chain	7	22255106790	8	0/100	0.00046
bits	4	3	chain	7	543067299	6	0/100	0.00046
bits	5	0	chain	-	failed (memory 3.5 GB / 900 s)	-	-	-
bits	5	3	chain	11	94122119296	7	0/39 (capped)	0.00171
bits	6	0	chain	-	failed (memory 3.5 GB / 900 s)	-	-	-
bits	6	3	chain	-	failed (memory 3.5 GB / 900 s)	-	-	-
isogeny	3	0	full	1	156829385	10	0/100	0
isogeny	4	0	full	9	0	0	0/100	0.00095
isogeny	5	0	full	18	0	0	1/100	0.00729
isogeny	6	0	full	25	0	0	2/100	0.01954
bits	4	0	full	7	0	0	0/100	0.00046
bits	4	3	full	7	1198241768	10	0/100	0.00046
bits	5	0	full	11	0	0	0/100	0.00171
bits	5	3	full	11	289898264927	11	0/19 (capped)	0.00171
bits	6	0	full	-	failed (memory 3.5 GB / 900 s)	-	-	-

form	t	G	mode	columns	multiply-adds / target	solving degree	hits / targets	coverage
bits	6	3	full	-	failed (memory 3.5 GB / 900 s)	-	-	-

Coverage for bases that no cell could run (`analysis/coverage.jsonl`): isogeny t = 7: 0.298 (65 columns), bits t = 7: 0.218, bits t = 8: 0.846. These are the sizes where m = 3 would give a hit per target or two, with systems of 21 to 24 tower variables.

Notes on the rows. - `failed`: the process exceeded the 3.5 GB address-space limit (`ulimit -v`, set because the isogeny t = 6 chain run reached 6.7 GB RSS before I stopped it) or the 900 s timeout; the harness keeps no stderr, so the two are not distinguished. Reported as intractable. - Capped rows (isogeny 5 chain: 1 target in 703 s; bits 5 G3 chain: 39 targets; bits 5 G3 full: 19 targets) are analysed on the targets done. - Full-mode rows with “all 100 undecided; F4 work 0”: F4 stopped before elimination (system over the oversize limit) and the oracle handed the system to the root grid. Their 1 and 2 hits (isogeny t = 5, 6) were read by the uncharged grid, so they are **not** F4 relations and carry no F4 cost. No per-relation figure is given. - Chain with G = 3 is far cheaper per target than G = 0 (bits t = 4: 5.4e8 vs 2.2e10) but coverage stays 0.00046, so per-relation stays near 1e12.

Comparison with m = 2 (preregistered)

Ladder data (`research/curve25519_algebraic_ladder_20261102/analysis/f4cost.jsonl`, m = 2, full): isogeny t = 6: 5.55e7 per target, 1 hit in 200 targets; t = 7: 7.46e8, 4/200; t = 8: 6.98e9, 7/200. Per-relation from the observed hit rate: 1.1e10 (t = 6), 3.7e10 (t = 7), 2.0e11 (t = 8). The ~6e9 quoted in the task comes from the ladder’s lambda_2 = 0.0095, which counts 2-sums over all 8 torsion cosets; for subgroup targets I measure the exact m = 2 coverage with the same filter as above: 0.0013 (t = 6, 172 of 1251 sums) and 0.0091 (t = 7), so per-relation at exact coverage is 4.2e10 (t = 6) and 8.2e10 (t = 7). I flag this as a possible overstatement of lambda_2 by a factor 8 in the ladder’s hit-rate model for subgroup targets (the observed 1/200 is consistent with 0.0013 and with 0.0095 at this sample size); it does not change the conclusion.

quantity	per-relation multiply-adds
best m = 2 (task / ladder lambda_2)	~6e9
best m = 2, observed hits (isogeny t = 6)	1.1e10
best m = 2, exact subgroup coverage	4.2e10
best m = 3 (bits t = 4, G = 3, chain, exact coverage)	1.0e12
next m = 3 (isogeny t = 4 chain)	1.8e13

Preregistered rule: improvement needs C < B/2 with >= 5 hits; match B/2..2B; regression C > 2B. C = 1e12 is 170x B = 6e9 (24x the exact-coverage m = 2 figure). **Regression.** The rho gap is not computed here (no rho reference run belongs to this experiment; no leaderboard row is claimed).

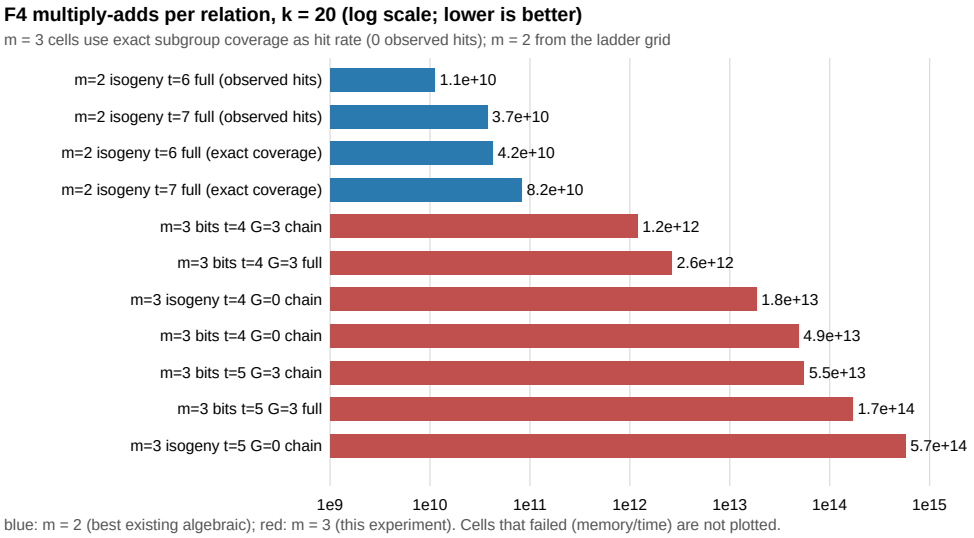
Why

The chained system has 3t + 1 variables; the number of F4 multiply-adds grows by about 65x then 240x per extra tower level at t = 3..5 (2.7e8, 1.7e10, 4.2e12), while coverage grows by about 8x per level. The m = 2 cells have 2t variables and a degree-5 solving degree throughout; m = 3 chains need degree 6 to 8, and the full presentation needs 10 to 11 (or exceeds the size limit). Reaching coverage 0.02 needs t = 6 (25 columns, 19 variables), which is out of reach of this solver at 3.5 GB.

Deviations

- The preregistered per-relation = cost / hit rate cannot use measured hit rates (all zero where F4 ran); I used the exact coverage as the hit rate and labelled it. This was added after data.
- Memory cap 3.5 GB per process and 900 s hard timeout were added (coordinator instruction); the first run of `isogeny t = 6 chain` was stopped manually at 6.7 GB and re-run under the cap (recorded `failed`).
- `f4cost` JSON gained fields (`m`, `mode`, `coverage_exact`, `covered_points`, `covered_points_all_cosets`, `muladds_per_relation*`); the coverage rows were taken by a rebuilt binary (targets = 0) after the cells ran; solving code is unchanged between the two builds.
- Preliminary test runs before the coordinator's note on the shared build directory were discarded; all quoted results come from the exclusive build.

Figure



per-relation cost, m = 3 vs m = 2

Values: analysis/per_relation.tsv (jq over `analysis/f4cost_m3.jsonl`, `analysis/coverage.jsonl` and the ladder's `f4cost.jsonl`). Report PDF: REPORT.pdf.

Coordinator review note

The λ_2 remark above was checked against the ladder code (`support` in `src/bin/curve25519_family.rs`): λ_2 counts 2-sums of the **8-projected** base, so it is the yield of a cofactor-aware oracle, not of the `pkm-semaev` oracle as run (which decomposes R into unprojected tower points). The ladder README already labels its per-λ figure a lower bound for this reason; lead 2 (cofactor-aware algebraic oracle) targets that factor ≈ 8. The m = 3 verdict holds under every m = 2 reading (observed, exact coverage, λ_2).